

Exercise 4.1: Generic ArrayList Reversal

Exercise41.java:

```
import java.util.ArrayList;

public class Exercise41 {
    public static <E> void reverse(ArrayList<E> list) {
        for (int i = 0, j = list.size() - 1; i < j; i++, j--) {
            E temp = list.get(i);
            list.set(i, list.get(j));
            list.set(j, temp);
        }
    }

    public static void main(String[] args) {
        ArrayList<String> words = new ArrayList<>();
        words.add("Java");
        words.add("Generics");
        words.add("Are");
        words.add("Useful");

        System.out.println("Before reverse: " + words);
        reverse(words);
        System.out.println("After reverse:  " + words);
    }
}
```

Expected Output:

```
Before reverse: [Java, Generics, Are, Useful]
After reverse:  [Useful, Are, Generics, Java]
```

Exercise 4.2: Generic ArrayList Shuffling

```
import java.util.ArrayList;

public class Exercise4_2 {
    public static <E> void shuffle(ArrayList<E> list) {
        for (int i = list.size() - 1; i > 0; i--) {
            int j = (int) (Math.random() * (i + 1));

            E temp = list.get(i);
            list.set(i, list.get(j));
            list.set(j, temp);
        }
    }

    public static void main(String[] args) {
        ArrayList<Integer> list = new ArrayList<>();
        list.add(10);
        list.add(3);
        list.add(25);
        list.add(8);
        list.add(42);

        System.out.println("Before shuffling: " + list);
        shuffle(list);
        System.out.println("After shuffling: " + list);
    }
}
```

Exercise 4.3: Comparable and Generic

Task 1:

```
import java.util.ArrayList;

public class Exercise43 {
    public static <E extends Comparable<E>> E max(ArrayList<E> list) {
        if (list == null || list.isEmpty()) {
            return null;
        }

        E largest = list.get(0);

        for (int i = 1; i < list.size(); i++) {
            if (list.get(i).compareTo(largest) > 0) {
                largest = list.get(i);
            }
        }

        return largest;
    }

    public static void main(String[] args) {
        ArrayList<Integer> numbers = new ArrayList<>();
        numbers.add(12);
        numbers.add(7);
        numbers.add(25);
        numbers.add(3);

        System.out.println("List: " + numbers);
        System.out.println("Largest element: " + max(numbers));
    }
}
```

Task 2 - Plain-English Explanation:

The method `max` takes an `ArrayList` of **elements whose type implements Comparable to objects of the same type** and returns **the largest element in that list**.

Exercise 4.4: Multiple Generics

The original `Pair<E>` requires both `first` and `second` to have the same type. To allow different types, revise it to use **two** type parameters.

```
public class Pair<F, S> {
    public F first;
    public S second;

    public Pair(F e, S f) {
        first = e;
        second = f;
    }

    public static void main(String[] args) {
        Pair<Integer, Double> p1 = new Pair<>(1, 85.5);
        Pair<Integer, String> p2 = new Pair<>(2, "good");

        print(p1);
        print(p2);
    }

    public static void print(Pair<?, ?> p) {
        System.out.println(p.first + " " + p.second);
    }
}
```

Expected Output:

```
1 85.5
2 good
```

This solution keeps the structure very close to the given code, but generalizes it from one type parameter to two.

Exercise 4.5: Wildcard

Why the original code does not compile:

Although `Integer` is a subclass of `Object`, `ArrayList<Integer>` is not a subclass of `ArrayList<Object>`. Generic types in Java are invariant. This is exactly the kind of problem wildcards are meant to solve.

One Correction:

Change only the parameter type of `print` to use an unbounded wildcard:

```
import java.util.*;

public class week4test1 {
    public static void main(String[] args) {
        ArrayList<Integer> c = new ArrayList<>();
        c.add(3);
        c.add(4);
        c.add(12);
        print(c);
    }

    public static void print(ArrayList<?> o) {
        for (Object e : o) {
            System.out.println(e);
        }
    }
}
```

This is the cleanest correction because the method only needs to read and print elements; it does not need to add elements to the list.

Exercise 4.6: Wildcard Copying of Elements Between Lists

Revised Method:

```
import java.util.ArrayList;

public class Exercise46 {
    public static <T> void copy(ArrayList<? extends T> source,
                                ArrayList<? super T> target) {
        for (T item : source) {
            target.add(item);
        }
    }

    public static void main(String[] args) {
        ArrayList<Integer> source = new ArrayList<>();
        source.add(1);
        source.add(2);
        source.add(3);

        ArrayList<Number> target = new ArrayList<>();
        copy(source, target);

        System.out.println(target);
    }
}
```

Expected Output:

```
[1, 2, 3]
```

Exercise 4.7: Conceptual Understanding of Generics

1. What is the main advantage of using generics in Java?

The main advantage is **type safety at compile time**. Generics help detect type errors during compilation instead of waiting for runtime failures. They also improve readability because fewer casts are needed.

2. Why does Java require generic type arguments to be reference types rather than primitive types?

Short answer: Java generics work only with reference types because they are based on **type erasure** and are designed to operate with objects. Primitive types like `int` and `double` are not objects, so they cannot be used directly as type arguments. Instead, Java uses wrapper classes such as `Integer` and `Double`.

Long answer: Java generics require reference types because generics are implemented using **type erasure**. This means the compiler removes most generic type information after compilation and treats generic code in a uniform way at runtime. Primitive types such as `int` and `double` are not objects and do not fit into that object-based generic mechanism. Allowing primitives directly in generics would have required a different implementation model, likely creating separate specialized versions of generic classes and methods for each primitive type, which Java did not do when generics were introduced.

3. What is the difference between `?`, `? extends T`, and `? super T`?

- `?` means **any type at all**.
- `? extends T` means **T or any subclass of T**.
- `? super T` means **T or any superclass of T**.

4. Why is `GenericStack<Integer>` not a subclass of `GenericStack<Number>`, even though `Integer` is a subclass of `Number`?

Because Java generic types are **invariant**. Even if `Integer` extends `Number`, that relationship does not automatically carry over to `GenericStack<Integer>` and `GenericStack<Number>`.

5. What is type erasure, and what effect does it have at runtime?

Type erasure means the compiler checks generic type usage at compile time and then removes most generic type information afterward. At runtime, the program does not retain the full generic type details in the way beginners often expect. This supports backward compatibility with older Java code.

6. Why is `T value = new T();` not allowed in Java?

Because after type erasure, Java does not know what concrete type `T` is at runtime, so it cannot directly create an object with `new T()`. The compiler therefore does not allow it. This is one of the restrictions caused by type erasure.